Hafta 5 — Tag Helper'lar

> **Ders:** Web Programlama · **Hafta:** 5 / 13 ders haftası · **Süre:** 3 × 50 dakika **Ön koşul:** Hafta 4 (form, `name`, Model Binding, PRG) ve Hafta 3 (Razor, `_ViewImports`) **Bu dosya sınav kapsamındadır.** Daha derin konular `H05-detay.md` dosyasındadır ve **sınav kapsamı dışıdır.**

🎯 Bu Haftanın Hedefleri

Bu haftanın sonunda:

1. Bir Tag Helper'ın **ne olduğunu** ve `@Html` yardımcılarından **farkını** açıklayabileceksiniz.
2. `@addTagHelper` satırının ne işe yaradığını ve **olmadığında ne olduğunu** gösterebileceksiniz.
3. `asp-controller`, `asp-action`, `asp-route-*` ve Learn'deki diğer Anchor attribute'leriyle **URL üretebilecek**, elle yazılan `href`'in neden kırıldığını **anlatabileceksiniz**.
4. `asp-for`'un ürettiği `name`, `id`, `type` ve `value` değerlerinin **nereden geldiğini** gösterebileceksiniz.
5. `<label asp-for>`, `<textarea asp-for>` ve `<select asp-items>` etiketlerini **kullanabileceksiniz**.
6. `[Display]` ve `[DataType]` attribute'lerinin üretilen HTML'e **nasıl yansıdığını** gösterebileceksiniz.
7. **CSRF** saldırısının nasıl çalıştığını **anlatabileceksiniz**.
8. Tag Helper formunun ürettiği **anti-forgery token**'ı bulabilecek ve `[ValidateAntiForgeryToken]` ile **doğrulatabileceksiniz**.
9. Token'ın neden GET'te değil **POST'ta** gerektiğini açıklayabileceksiniz.
10. Hafta 4'te **sessiz** kalan yazım hatasının bu hafta **derlemede** yakalandığını gösterebileceksiniz.
11. `<environment>`, `<cache>`, `asp-append-version`, `asp-fallback-*` ve `<partial for>`'un **ne ürettiğini** gösterebileceksiniz.
12. **Kendi Tag Helper'ınızı** yazıp adlandırma kuralını, `Process`'i ve `TagMode`'u açıklayabileceksiniz.

🧩 **Haftanın ilkesi — Çerçeve, doğru olanı kolay yapar**

> Tag Helper bir kısaltma değildir. Yazmadığınız `name`'i, yazmadığınız `action`'ı ve **hiç istemediğiniz güvenlik alanını** sizin yerinize üretir. Bir aracın değeri, size yazdırdığı satırlarda değil, **yazmayı unutamayacağınız hâle getirdiği** satırlardadır.

---

1. Tag Helper Nedir?

1.1. Aslında üç haftadır kullanıyorsunuz

Hafta 2'den beri her görünümde şuna benzer satırlar var:

```
<a asp-controller="Kitap" asp-action="Index">Kitaplar</a>
```

Bu bir HTML etiketi **gibi görünüyor**, ama `asp-controller` diye bir HTML attribute'ü **yoktur**. Tarayıcıya giden şey şu:

```
<a href="/Kitap">Kitaplar</a>
```

Arada bir şey oldu: sunucu, etiketi tarayıcıya göndermeden önce **işledi**. O şeyin adı **Tag Helper**.

> Hafta 3'te `ViewData["Title"]` 'ı, Hafta 4'te `HttpContext` 'i böyle tanımıştık: yeni bir şey öğrenmiyoruz, **zaten kullandığımız şeyin adını koyuyoruz.**

Tanıdığınız başka Tag Helper'lar da var:

Zaten kullandığınız	Ne zamandan beri
`<a asp-controller="…" asp-action="…">`	Hafta 2
`<partial name="_KitapKart" model="k" />`	Hafta 3
`<vc:ziyaret-rozeti />`	Hafta 3
`<link … asp-append-version="true" />`	Proje şablonundan beri

Sonuncusunu bölüm 1.6'da açıyoruz; çatının hazır Tag Helper'larının tam listesi bölüm 1.5'te.

1.2. HTML gibi görünen C#

Bir Tag Helper, sunucuda çalışan bir **C# sınıfıdır**. Görevi: bir etiketi yakalamak, attribute'lerine bakmak ve yerine **düz HTML** koymak.

```
Yazdığınız                          Çatı çalışır              Tarayıcıya giden
──────────────────────────────      ──────────────────────    ──────────────────────

<a asp-action="Index">Liste</a>  →  AnchorTagHelper        →  <a href="/Kitap">Liste</a>
<input asp-for="Baslik" />       →  InputTagHelper         →  <input type="text"
                                                               id="Baslik"
                                                               name="Baslik" ... />
```

Buradan **teşhis kuralı** çıkar:

> **Çalışan bir Tag Helper, kendi attribute'ünü siler.** Tarayıcıda (Ctrl+U) `asp-` ile başlayan bir attribute **görüyorsanız**, o Tag Helper **çalışmamıştır**.

Bu kural, bölüm 1.3'teki hatayı saniyeler içinde teşhis etmenizi sağlar. Bölüm 5'te böyle bir sınıfı kendimiz yazacağız.

1.3. `@addTagHelper` — nereden geliyorlar?

Çatı, hangi sınıfların Tag Helper sayılacağını kendiliğinden bilmez. Bunu `Views/_ViewImports.cshtml` dosyasındaki satır söyler:

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Okunuşu: "*`Microsoft.AspNetCore.Mvc.TagHelpers` derlemesindeki (`*`) bütün Tag Helper'ları bu klasördeki görünümlerde kullan.*"

Hafta 3'te projenin **kendi** bileşenleri için ikinci bir satır eklemiştik:

```
@addTagHelper *, KampusKitap
```

`<vc:ziyaret-rozeti />` etiketini çalıştıran satır buydu.

🦎 **Satır olmazsa ne olur?** Hata **almazsınız**. Sayfa **200** döner. Ve hiçbir şey çalışmaz:

```
<!-- @addTagHelper olmadan tarayıcıya giden -->
<input asp-for="Metin" />
<a asp-controller="Kitap" asp-action="Index">bağlantı</a>
<form asp-action="Index" method="post">
```

Sonuçlar ağırdır:

Ne olmadı	Sonucu
`<input>` `name` almadı	Hafta 4'ten biliyoruz: `name`'i olmayan alan sunucuya **hiç ulaşmaz**
`<a>` `href` almadı	Bağlantı tıklanabilir bile değil
`<form>` `action` almadı	Form kendi adresine gönderir
Token üretilmedi	Bölüm 4'ün konusu

🧪 `/Etiket/Eksik` sayfası bunu canlı gösterir. Sayfa `@removeTagHelper` ile Tag Helper'ları bilerek kapatıyor — yani `@addTagHelper` unutulmuş gibi davranıyor.

1.4. `@Html` yardımcılarından farkı

Tag Helper'lardan önce aynı iş `@Html` metotlarıyla yapılıyordu. İkisi de **aynı HTML'i** üretir; fark **yazım biçimindedir**:

```
@* HTML Helper — bir C# metot çağrısı *@
@Html.TextBoxFor(m => m.Baslik, new { @class = "form-control", size = 40 })

@* Tag Helper — bir HTML etiketi *@
<input asp-for="Baslik" class="form-control" size="40" />
```

	`@Html`	Tag Helper
Görünüm	C# metodu	HTML etiketi
HTML attribute eklemek	Anonim nesne: `new { @class = "…" }`	**Doğrudan yazılır**
`class` yazımı	`@class` (çünkü `class` C# anahtar kelimesi)	`class`
Editör yardımı	C# tamamlama	**HTML tamamlama**
Tasarımcı okuyabilir mi?	Zor	**Evet** — dosya HTML'e benziyor

`@Html` **kaldırılmadı**; hâlâ çalışır ve bazı işler yalnızca onunla yapılır (`Html.GetEnumSelectList<T>()` gibi — bölüm 3.4'te kullanacağız). Ama form yazarken tercih edilen yol Tag Helper'dır.

1.5. Çatıdaki 18 Tag Helper

Microsoft Learn'ün *built-in Tag Helpers* listesinde 18 Tag Helper var. Hepsi aynı makinedir: `TagHelper` sınıfından türeyen, `Process` metodu olan birer C# sınıfı. Bu hafta 14'ünü göreceğiz:

Aile	Tag Helper'lar	Nerede
URL üretenler	Anchor, Form Action	Bölüm 2
Form	Form, Input, Label, Select, Textarea	Bölüm 3 ve 4
Sayfa düzeyi	Environment, Cache, Distributed Cache, Image, Link, Script, Partial	Bölüm 1.6
Doğrulama	Validation Message, Validation Summary	Ürettikleri HTML bölüm 3.6'da; işleri **Hafta 6**
Blazor	Component, Persist Component State	Bu dersin kapsamı dışında

Listede olmayan bir tane daha var: adreslerin başındaki `~/` 'yi uygulamanın köküne çeviren iç Tag Helper. `src="~/favicon.ico"` yazdığınızda tarayıcıya `src="/favicon.ico"` gitmesini o sağlar. Bölüm 5'te 19.'yu biz yazacağız.

1.6. Sayfa düzeyindeki Tag Helper'lar

Formdaki Tag Helper'lar bir **model özelliğini** HTML'e çevirir. Buradakiler ise sayfanın **hangi ortamda**, **ne kadar süre** ve **hangi dosyayla** çizileceğine karar verir. Bu bölümdeki bütün çıktılar `/Etiket/Sayfa` sayfasında ölçüldü.

`<environment>` — ortama göre üretmek

```
<environment include="Development"> … </environment>
<environment exclude="Development"> … </environment>
<environment names="Staging,Production"> … </environment>
```

Ortam	`include="Development"`	`exclude="Development"`	`names="Staging,Production"`
Development (`dotnet run`)	üretildi	—	—

Production	—	üretildi	üretildi

Koşulu tutmayan blok CSS ile gizlenmez; **hiç üretilmez**, Ctrl+U'da da yoktur. Ortamı değiştirmek için:

```
dotnet run -e ASPNETCORE_ENVIRONMENT=Production
```

> ⚠️ .NET 10'da `dotnet run --environment Production` **işe yaramaz**: SDK'nın `-e` / `--environment` seçeneği `AD=DEĞER` biçiminde bir ortam değişkeni bekler ve uygulama Development'ta kalır.

`<cache>` — aynı parçayı her istekte yeniden çizmemek

```
<cache expires-after="@TimeSpan.FromSeconds(10)">
    @DateTime.Now.ToString("HH:mm:ss")
</cache>
```

F5'e dört saniye arayla basıldığında ölçülen değerler:

İstek	Cache dışında	Cache içinde
1	10:26:37	10:26:37 — yeni çizildi
2	10:26:43	10:26:37 — bellekten
3	10:26:47	10:26:47 — 10 saniye doldu, yeniden çizildi
4	10:26:51	10:26:47 — bellekten

`vary-by-query="kategori"` eklenirse her `kategori` değeri için **ayrı** bir kopya tutulur; `vary-by-route`, `vary-by-user` ve `vary-by-cookie` aynı mantıkla çalışır. Kopya sunucunun belleğindedir ve uygulama yeniden başlayınca silinir. Birden çok sunucunun paylaştığı kopya için `<distributed-cache name="…">` vardır: anahtarı `name` attribute'üdür ve kopyayı `Program.cs`'te kaydedilen depoya (Redis, SQL Server) yazar.

`asp-append-version` — tarayıcı eski dosyayı göstermesin

```
<img src="~/favicon.ico" asp-append-version="true" alt="…" />
```

```
<img src="/favicon.ico?v=Jtxf9L-5ITKRc1gIRl4VbUpGkRNfOBXjYTdhJD4facM" alt="…" />
```

`?v=…` dosyanın **içeriğinden** hesaplanır. Dosya değişince adres de değişir; tarayıcı adresi farklı gördüğü için eski kopyayı kullanmaz. `_Layout.cshtml`'deki `site.css` ve `site.js` satırları bu attribute'ü proje şablonundan beri taşıyor: "CSS'i değiştirdim ama tarayıcı eskisini gösteriyor" sorununun hazır çözümü budur.

`asp-fallback-*` — asıl dosya gelmezse yedeği

Gerçek projede asıl adres bir CDN'dir, yedek kendi sunucunuzdaki kopyadır. Örnek projede asıl adres **bilerek yanlış** verildi; böylece yedek internetsiz de görülür:

```
<link rel="stylesheet" href="~/css/olmayan.css"
      asp-fallback-href="~/css/yedek-demo.css"
      asp-fallback-test-class="yedek-test"
      asp-fallback-test-property="position"
      asp-fallback-test-value="absolute" />
<script src="~/js/olmayan.js"
        asp-fallback-src="~/js/yedek-demo.js"
        asp-fallback-test="window.yedekDemo"></script>
```

Tag Helper asıl etiketin arkasına küçük bir **sınama betiği** ekler. CSS için gizli bir `<meta>` etiketine `yedek-test` sınıfını verir ve `position` değeri `absolute` mu diye bakar; değilse asıl dosya gelmemiştir ve betik yedeği sayfaya yazar. JS için `window.yedekDemo` tanımlı mı diye bakar. Tarayıcının Network sekmesinde:

İstek	Durum
`/css/olmayan.css` · `/js/olmayan.js`	**404**
`/css/yedek-demo.css` · `/js/yedek-demo.js`	**200** — sınama başarısız oldu, yedek yüklendi

`<partial>` — `model` mi, `for` mu?

Hafta 3'te `<partial name="…" model="…">` yazdık. Partial'ın içinde `asp-for` varsa iki yazım **farklı** `name` üretir. `_KitapAlan.cshtml`'de yalnızca `<input asp-for="Baslik" />` var:

Yazım	Üretilen
`<partial name="_KitapAlan" model="Model.OneCikan" />`	`id="Baslik" name="Baslik"`
`<partial name="_KitapAlan" for="OneCikan" />`	`id="OneCikan_Baslik"` `name="OneCikan.Baslik"`

`model=` yalnızca **veriyi** verir; partial kendini sayfanın en üstünde sanır. `for=` veriyle birlikte **yolu** da verir; form gönderildiğinde Model Binding değeri `OneCikan.Baslik`'a yazabilir — bölüm 3.5'teki noktalı ad kuralı.

---

2. URL Üreten Tag Helper'lar

2.1. `asp-controller` ve `asp-action`

```
<a asp-controller="Kitap" asp-action="Detay">Detay</a>
```

Çatı bu ikisini alır, **rota tablosuna sorar** ("Kitap/Detay'a hangi adresten gidiliyor?") ve çıkan adresi `href` olarak yazar.

`asp-controller` yazılmazsa **içinde bulunduğunuz controller** varsayılır:

```
@* Etiket controller'ının görünümünde *@
<a asp-action="Index">Ana sayfa</a>      →      <a href="/Etiket">Ana sayfa</a>
```

2.2. `asp-route-*` — yola mı, query string'e mi?

Ek değerler `asp-route-` öneki ile verilir. Nereye yazılacağına **rota şablonu** karar verir:

```
<a asp-controller="Kitap" asp-action="Detay" asp-route-id="3">…</a>
<a asp-controller="Kitap" asp-action="Ara"   asp-route-kelime="suc">…</a>
```

```
<a href="/Kitap/Detay/3">…</a>
<a href="/Kitap/Ara?kelime=suc">…</a>
```

Aradaki farkın sebebi rota şablonudur:

```
pattern: "{controller=Home}/{action=Index}/{id:int?}"
```

- `id` şablonda **var** → değer **yola** yazıldı.
- `kelime` şablonda **yok** → değer **query string'e** düştü.

İkisi birlikte de verilebilir:

```
asp-route-id="3" asp-route-kaynak="menu"   →   /Kitap/Detay/3?kaynak=menu
```

> Hafta 4'te Model Binding'in bu değerleri **okuma** sırasını görmüştük (Form → Rota → Query string). `asp-route-*` aynı tablonun **yazma** yönüdür.

2.3. Elle yazılan `href` neden kırılır?

Bugün iki yazım **aynı** çıktıyı veriyor:

```
<a asp-controller="Kitap" asp-action="Detay" asp-route-id="3">Kitap</a>
<a href="/Kitap/Detay/3">Kitap</a>
```

Her ikisi de `<a href="/Kitap/Detay/3">`. Fark, rota değiştiğinde ortaya çıkar. `Program.cs`'te şablonun başına bir önek ekleyelim:

```
pattern: "site/{controller=Home}/{action=Index}/{id:int?}"
```

Yazım	Rota değişince	Sonuç
`asp-action` ile üretilen	`/site/Kitap/Detay/3` olur	**200**
Elle yazılan `href`	`/Kitap/Detay/3` kalır	**404**

Tuzağın asıl kötü yanı sessiz olmasıdır. Derleyici uyarmaz, uygulama açılır, sayfa görünür. Bağlantının kırık olduğu yalnızca **tıklandığında** anlaşılır — ve genellikle tıklayan siz olmazsınız.

> 🧪 `/Etiket/Baglanti` sayfası Tag Helper'la üretilen bağlantıları elle yazılanın yanına basar; `Program.cs`'teki tek satırı değiştirip sayfayı yenilediğinizde Tag Helper'la üretilenler taşınır, elle yazılan kırılır.

2.4. Form etiketinde aynı mantık

```
<form asp-controller="Kitap" asp-action="EkleTH" method="post">
```

```
<form method="post" action="/Kitap/EkleTH">
```

`action` adresi rotadan üretildi. Ama Form Tag Helper'ı **bir şey daha** yaptı; onu bölüm 4'te göreceğiz.

2.5. Learn'deki diğer Anchor attribute'leri

Microsoft Learn'ün *Anchor Tag Helper* sayfası bütün örneklerini tek bir `SpeakerController` üzerinden verir. Örnek projede aynı controller **adları değiştirilmeden** kuruldu; hepsi `/Speaker` sayfasında canlı:

```
[Route("Speaker/{id:int}")]
public IActionResult Detail(int id) => …

[Route("/Speaker/Evaluations", Name = "speakerevals")]
public IActionResult Evaluations() => …

[Route("/Speaker/EvaluationsCurrent", Name = "speakerevalscurrent")]
public IActionResult Evaluations(int speakerId, bool currentYear) => …
```

Yazım	Ölçülen `href`	Ne yaptı
`asp-route="speakerevals"`	`/Speaker/Evaluations`	**Adlı route**: controller ve action yazılmadı
`asp-route="speakerevalscurrent"` `asp-all-route-data="parms"`	`/Speaker/EvaluationsCurrent?speakerId=11&currentYear=true`	Sözlükteki her çift bir değer oldu
`asp-action="Evaluations"` `asp-fragment="SpeakerEvaluations"`	`/Speaker/Evaluations#SpeakerEvaluations`	Sayfanın içindeki başlığa gider

`asp-protocol="https"` `asp-controller="Home"` `asp-action="About"`	`https://localhost:5072/Home/About`	**Tam** adres, isteğin host'uyla
`asp-protocol="https"` `asp-host="microsoft.com"` …	`https://microsoft.com/Home/About`	Host da verildi

`parms` görünümde kurulan bir sözlüktür: `new Dictionary<string, string> { { "speakerId", "11" }, { "currentYear", "true" } }`. Learn'de iki attribute daha var: `asp-area` bir **Area** (`/Blogs/Home/AboutBlog` gibi), `asp-page` ise **Razor Pages** ister; örnek projede ikisi de yok.

Learn ile ölçüm arasındaki fark. Belge `asp-action="Detail"` ve `asp-route-id` için `/Speaker/Detail/12` yazıyor. Bu projede ölçülen `/Speaker/10`: action'ın üzerindeki `[Route("Speaker/{id:int}")]` şablonu kullanıldı. Tablonun devamı daha öğretici:

Yazım	Üretilen	Durum
`asp-action="Detail" asp-route-id="10"`	`/Speaker/10`	200
`asp-action="Detail" asp-route-speakerid="10"`	`/Speaker/Detail?speakerid=10`	**404**
`asp-controller="Kitap" asp-action="Detya"` (yazım hatası)	`/Kitap/Detya/3`	**404**

Tag Helper adresi şablondan kurar; o adreste çalışan bir action olup olmadığını denetlemez. Bölüm 2.3'teki rota değişikliğine karşı korur, ama action adındaki yazım hatasına karşı korumaz. `asp-for`'daki yazım hatası derlemede yakalanır (bölüm 3.1), çünkü orada bir C# ifadesi yazıyoruz; `asp-action`'a ise **düz metin** veriyoruz.

2.6. Bir form, iki düğme — Form Action Tag Helper

Anchor'ın attribute'leri `<button>` ve `<input type="image">` üzerinde de çalışır. Bu kez adres `href`'e değil, düğmenin `formaction`'ına yazılır ve formun `action`'ını ezer: form, hangi düğmeye basıldıysa oraya gönderilir.

```
<form method="post">
    <input name="mesaj" value="Merhaba" />
    <button asp-action="Csrf">Kaydet</button>
    <button asp-controller="Speaker" asp-action="Index">Learn örneklerine</button>
</form>
```

```
<form method="post">
    <input name="mesaj" value="Merhaba" />
    <button formaction="/Etiket/Csrf">Kaydet</button>
    <button formaction="/Speaker">Learn örneklerine</button>
    <input name="__RequestVerificationToken" type="hidden" value="CfDJ8…" />
</form>
```

Formun kendi `action`'ı yok, ama `method="post"` olduğu için Form Tag Helper yine de devrede ve token'ı ekledi (bölüm 4.2). Birinci düğme formu korumalı `/Etiket/Csrf`'ye götürür ve token doğrulanır; ikinci düğme **aynı formu** `/Speaker`'a götürür.

3. Form Tag Helper'ları

Hafta 4'te formu **elle** yazdık:

```
<input type="text" id="baslikAlani" name="Baslik" value="@Model.Baslik" size="40" />
```

Burada **üç** şeyden biz sorumluyduk: `name`'in sınıftaki özellik adıyla aynı olması, `value`'nun modelden gelmesi, `type`'ın alana uygun olması. Üçünde de hata yapabilir ve **uyarı almazdık**.

3.1. `asp-for` — tek attribute, dört çıktı

```
<input asp-for="Baslik" size="40" />
```

```
<input size="40" type="text" data-val="true"
       data-val-required="The Başlık field is required."
       id="Baslik" name="Baslik" value="" />
```

Tek bir `asp-for`, dört ayrı soruyu cevapladı:

Çıktı	Nereden geldi
`name="Baslik"`	Özellik **adından**
`id="Baslik"`	Özellik adından
`type="text"`	Özellik **tipinden** (`string`)
`value=""`	Modeldeki **değerden**

Yazdığımız `size="40"` **olduğu gibi korundu**: Tag Helper tanımadığı attribute'lere dokunmaz.

Kazanç yazım tasarrufu değildir. Asıl kazanç şu:

```
<input asp-for="Balsik" />     @* yazım hatası *@
```

```
error CS1061: 'KitapEkleVM' does not contain a definition for 'Balsik'
```

Proje **derlenmez**. Hafta 4'te aynı hata sayfayı sessizce boş bırakıyordu.

3.2. `type` tipten çıkarılıyor

C# tipi	Üretilen `type`
`string`	`text`

`int`, `int?`	`number`
`decimal`	**`text`** ← aşağıya bakın
`DateTime`	`datetime-local`
`bool`	`checkbox`
`[DataType(DataType.Password)]`	`password` — ve **`value` yazılmaz**
`[DataType(DataType.EmailAddress)]`	`email`

`decimal` neden `number` değil? Sunucu Türkçe kültürde çalıştığı için fiyat `120,50` biçiminde — **virgülle** — yazılır. HTML'in `type="number"` alanı ise yalnızca **noktayı** kabul eder; virgüllü değeri gösteremez. Çatı, kullanıcının yazdığını kaybetmemek için `text` tercih eder. Bu, Hafta 4'te gördüğümüz ondalık ayracı sorununun (`H04-detay.md` bölüm 5) HTML tarafındaki yüzüdür.

Değerin **biçimini** siz seçmek isterseniz `asp-format` vardır: `<input asp-for="Fiyat" asp-format="{0:N4}" />` → `value="120,5000"`. Ölçerken bir ayrıntı daha çıktı: aynı alan sayfada ikinci kez çizildiğinde `data-val-*` attribute'leri **tekrar yazılmaz**; çatı onları bir alan için yalnızca bir kez üretir.

`bool` iki alan üretir. Onay kutusunun yanına **gizli** bir alan da konur:

```
<input type="checkbox" id="Onayli" name="Onayli" value="true" />
...
<input name="Onayli" type="hidden" value="false" />
```

Hafta 4'te "işaretlenmemiş onay kutusu gövdede **hiç gelmez**" demiş ve bunu bir Hata Köşesi satırı olarak bırakmıştık. Tag Helper açığı böyle kapatıyor: kutu işaretliyse `true`, değilse gizli alandan gelen `false` gönderilir.

3.3. `<label asp-for>` ve `[Display]`

```
<label asp-for="Baslik"></label>
```

```
<label for="Baslik">Başlık</label>
```

İki şey oldu: `for` değeri alanın `id`'siyle **eşleştirildi** (etikete tıklayınca alan odaklanır), metin ise **modelden** geldi.

Metni belirleyen şey `[Display]` attribute'üdür:

```
[Display(Name = "Fiyat (TL)")]
public decimal Fiyat { get; set; }
```

```
<label for="Fiyat">Fiyat (TL)</label>
```

`[Display]` **bir doğrulama attribute'ü değildir**; yalnızca gösterilecek adı belirler. Yazılmazsa özellik adının kendisi kullanılır.

3.4. `<select asp-items>` ve `SelectList`

```
<select asp-for="KategoriId"
        asp-items="@(new SelectList(KitapDeposu.Kategoriler, "Key", "Value"))">
    <option value="">— seçiniz —</option>
</select>
```

```
<select id="KategoriId" name="KategoriId">
    <option value="">— seçiniz —</option>
    <option value="1">Edebiyat</option>
    <option value="2">Bilgisayar</option>
    <option value="3">Bilim kurgu</option>
</select>
```

`SelectList`'in üç parçası vardır: **kaynak liste**, `value` olacak alanın adı, gösterilecek metnin alanı.

Beş ayrıntı:

1. **Elle yazdığımız `<option>` silinmez.** Tag Helper ürettiklerini onun **arkasına** ekler. "Hiçbiri" seçeneğini isteyen biziz; çatı onu kendiliğinden koymaz.
2. **Seçili olan kendiliğinden işaretlenir.** Modeldeki `KategoriId` 2 ise, çatı o seçeneğe `selected="selected"` yazar. Hafta 4'te bunu elle yapmak gerekiyordu.
3. **`enum` için hazır yardımcı var:** `asp-items="Html.GetEnumSelectList<KategoriTuru>()"`.
4. **Özellik bir liste ise** (`List<int>`), `<select>` kendiliğinden `multiple="multiple"` olur ve listedeki her değer `selected` işaretlenir.
5. **Seçenekleri gruplamak** için her `SelectListItem`'a bir `SelectListGroup` verilir; çatı `<optgroup label="Lisans">` üretir.

3.5. `<textarea asp-for>` ve iç içe nesneler

`<textarea asp-for="Aciklama"></textarea>` aynı mantıkla `name`, `id` ve içeriği üretir.

İç içe bir nesnenin özelliği için:

```
<input asp-for="Adres.Sehir" />
```

```
<input type="text" id="Adres_Sehir" name="Adres.Sehir" value="" />
```

`name` **noktalı,** `id` **alt çizgili.** Sebebi: Model Binding iç içe nesneyi `Adres.Sehir` biçiminde bekler (Hafta 4 bölüm 3.4), ama HTML `id` değerinde nokta CSS ve JavaScript tarafında sorun çıkarır. Çatı ikisini birden doğru yapar.

3.6. `data-val-*` nereden çıktı?

Üretilen HTML'de hiç yazmadığımız attribute'ler var:

```
data-val="true" data-val-required="The Başlık field is required."
```

`[StringLength(50, MinimumLength = 3)]` eklerseniz bunlara `data-val-length-max="50"`, `data-val-length-min="3"` ve ayrıca gerçek bir HTML attribute'ü olan `maxlength="50"` eklenir.

Bu hafta bunları okuyan hiçbir şey yok. Sayfada doğrulama betiği yüklü değil; attribute'ler orada duruyor ve hiçbir işe yaramıyor. Onları okuyan betik ve —asıl önemlisi— **sunucu tarafı doğrulama Hafta 6'nın konusudur.**

Doğrulama ailesinin iki Tag Helper'ı da bugün HTML üretir — **boş** yer tutucu olarak:

```
<span asp-validation-for="Metin"></span>
<div asp-validation-summary="All"></div>
```

```
<span class="field-validation-valid" data-valmsg-for="Metin" data-valmsg-replace="true"></span>
<div class="validation-summary-valid" data-valmsg-summary="true"><ul><li style="display:none"></li
```

İçlerini Hafta 6'da sunucunun bulduğu hatalar dolduracak.

Bir ayrıntı bugünden anlaşılmalı: `string` özelliği `data-val-required` aldı, `string?` **almadı**. Bunu yapan bir attribute değil, C#'ın **nullable reference types** özelliğidir: `string` "boş olamaz" demektir.

> 🧪 `/Etiket` sayfası her tip için üretilen HTML'i yan yana basar. Tabloyu elle yazmadık: sayfadaki küçük betik her denetimin kendi `outerHTML` 'ini okuyup altına koyuyor.

---

4. CSRF ve Tag Helper'ın Sessiz Katkısı

4.1. Saldırı nasıl çalışıyor?

Diyelim ki kullanıcı sitenizde oturum açtı; tarayıcısında bir oturum cookie'si var. Sonra başka bir sayfaya gidiyor. O sayfada görünmeyen bir form var:

```
<!-- kotuniyet.example sitesindeki sayfa -->
<form method="post" action="https://sizinsite.example/Hesap/ParaGonder">
    <input type="hidden" name="alici" value="saldirgan" />
    <input type="hidden" name="tutar" value="5000" />
</form>
<script>document.forms[0].submit();</script>
```

Kullanıcı hiçbir şey yapmadan form gönderilir. Ve **tarayıcı, sizin sitenizin cookie'sini bu isteğe otomatik ekler** — çünkü adres sizin sitenize gidiyor. Sunucu açısından istek **meşru** görünür.

Bu saldırının adı **CSRF** (Cross-Site Request Forgery).

🧩 Hafta 4'ün ilkesi burada bir adım ilerliyor: istemciden gelen **veri** güvenilir değildi; artık isteğin **kaynağı** da güvenilir değil.

4.2. Token nereden geliyor?

Savunma şudur: sunucu forma, **kendi ürettiği** gizli bir değer koyar ve POST'ta onu arar. Saldırganın sayfası bu değeri **bilemez**, çünkü başka bir sitenin sayfasının içeriğini okuyamaz.

Tag Helper bunu **siz istemeden** yapar:

```
<form asp-controller="Kitap" asp-action="EkleTH" method="post">
```

```
<form method="post" action="/Kitap/EkleTH">
    ...
    <input name="__RequestVerificationToken" type="hidden" value="CfDJ8A4-mf96z…" />
</form>
```

Aynı formu **elle** yazarsanız:

```
<form method="post" action="/Kitap/EkleTH">
```

```
<form method="post" action="/Kitap/EkleTH">
    ...
</form>
```

Token yok. Hafta 4'te yazdığımız bütün formlar bu durumdaydı.

4.3. `[ValidateAntiForgeryToken]` — üretmek yetmez

Token'ın **üretilmesi** tek başına hiçbir şey yapmaz; sunucudan onu **kontrol etmesini** istemek gerekir:

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult EkleTH(KitapEkleVM girdi) { … }
```

Üç durumu ölçelim:

İstek	Sonuç
Tag Helper formundan (token var, cookie eşleşiyor)	**302** — action çalıştı, PRG devrede
Token **yok**	**400 Bad Request**
Token var ama **cookie yok**	**400 Bad Request**

Reddedilen istekte dikkat edilecek üç şey var:

- Ekranda **hiçbir açıklama yok** — boş bir sayfa gelir.
- Bir **exception değildir**; geliştirici hata sayfası bile görmezsiniz.
- Action'ın içindeki kod **hiç çalışmamıştır**; istek oraya ulaşmadan reddedilir.

> **Son satır neden önemli?** Çünkü bu, doğru davranıştır. Kod çalışıp sonra "acaba meşru mu?" diye sormaz; istek **kapıdan içeri alınmaz**.

Her POST action'ına tek tek yazmak yerine uygulamanın tamamına uygulanabilir: `[AutoValidateAntiforgeryToken]`. Ayrıntısı `H05-detay.md` bölüm 3'tedir.

4.4. Token neden GET'te yok?

Aynı Tag Helper, `method="get"` ile yazıldığında **token üretmez**:

```
<form asp-action="Ara" method="get">   →   <form method="get" action="/Kitap/Ara">
```

Sebebi Hafta 4'ün kuralıdır: **okuma GET, değiştirme POST**. CSRF, kullanıcıya istemediği bir **değişikliği** yaptırma saldırısıdır. Veri değiştirmeyen bir GET isteğinin sahtesini göndermek saldırgana bir şey kazandırmaz.

> Bunun tersi de doğrudur: veri **değiştiren** bir işlemi GET'e koyarsanız, token koruması da devre dışı kalır. "Silme bağlantısı" olarak yazılan `<a href="/Kitap/Sil/3">` bu yüzden tehlikelidir.

4.5. Tek başına cookie neden yetmez?

Saldırganın sayfası ne yapabilir?

Yapabilir mi?	
Sizin sitenize form göndermek	**Evet** — HTML'i kendi sayfasına yazar
Tarayıcının cookie'sini ekletmek	**Evet** — tarayıcı bunu otomatik yapar
Token'ın değerini öğrenmek	**Hayır** — başka sitenin sayfasını okuyamaz

Koruma tam olarak bu boşluktan doğar: **cookie otomatik gider, gizli alan gitmez.** Sunucu ikisinin **eşleştiğini** arar. Bu yüzden yalnızca cookie göndermek yetmez — ölçtük, **400** aldık.

> 💉 `/Etiket/Csrf` sayfasında üç form yan yana duruyor: Tag Helper + POST, Tag Helper + GET, elle yazılmış POST. İkisi çalışır, biri 400 döner.

5. Kendi Tag Helper'ınızı Yazmak

Bölüm 1.2'de Tag Helper'ın "sunucuda çalışan bir C# sınıfı" olduğunu söyledik. Bu bölümde o sınıfı **biz** yazıyoruz. İlk üç örnek Microsoft Learn'ün *Author Tag Helpers* sayfasından birebir alındı; sınıflar `TagHelpers/` klasöründe, çıktıları `/Etiket/Ozel` sayfasında.

5.1. En küçük Tag Helper: `<email>`

```
using Microsoft.AspNetCore.Razor.TagHelpers;

public class EmailTagHelper : TagHelper
{
    private const string EmailDomain = "contoso.com";

    public string MailTo { get; set; } = "";

    public override void Process(TagHelperContext context, TagHelperOutput output)
    {
        output.TagName = "a";                                    // <email> → <a>
        var address = MailTo + "@" + EmailDomain;
        output.Attributes.SetAttribute("href", "mailto:" + address);
        output.Content.SetContent(address);
    }
}
```

```
<email mail-to="Support"></email>
```

```
<a href="mailto:Support@contoso.com">Support@contoso.com</a>
```

Üç kural tek sınıfta:

C#	Görünümde	Kural
`EmailTagHelper` sınıfı	`<email>` etiketi	`TagHelper` eki düşer, kalan **kebab-case** yazılır (`KategoriRozetiTagHelper` → `<kategori-rozeti>`)
`MailTo` özelliği	`mail-to` attribute'ü	PascalCase → **kebab-case**
`Process` metodu	—	Etiketi tarayıcıya gitmeden önce dönüştürür

Çıktıda `mail-to` **yok**: bir C# özelliğine bağlanan attribute çıktıya kendiliğinden yazılmaz. Sınıfın görünümlerde tanınması için yeni bir satır da gerekmedi — `_ViewImports.cshtml`'deki `@addTagHelper *, KampusKitap` Hafta 3'ten beri orada (bölüm 1.3). Sınıfın `public` olması ve `TagHelper`'dan türemesi yeter.

5.2. `TagMode` — kendini kapatan etiket

Aynı Tag Helper'ı kendini kapatan yazımla kullanalım:

```
<email mail-to="Support" />
```

```
<a href="mailto:Support@contoso.com" />
```

Yazı yok. `SetContent` çalıştı, ama çıktı da kendini kapattığı için içerik yazılmadı; tarayıcı bunu açık kalmış bir `<a>` sayar ve ekranda boş bir bağlantı kalır. Çözüm, çıktının biçimini açıkça seçmektir. Projenin `FiyatTagHelper`'ı bunu yapar (**TODO 5.8**):

```
public class FiyatTagHelper : TagHelper
{
    public decimal Deger { get; set; }
    public string ParaBirimi { get; set; } = "₺";

    public override void Process(TagHelperContext context, TagHelperOutput output)
    {
        output.TagName = "span";
        output.TagMode = TagMode.StartTagAndEndTag;      // <fiyat … /> yazılsa da
        output.Attributes.SetAttribute("class", "fiyat");
        output.Content.SetContent($"{Deger:N2} {ParaBirimi}");
    }
}
```

Yazılan	Üretilen
`<fiyat deger="@(120.50m)" />`	`<span class="fiyat">120,50 ₺</span>`
`<fiyat deger="99" para-birimi="$" />`	`<span class="fiyat">99,00 $</span>`

Ctrl+U'da ₺ işareti `₺` olarak görünür: `SetContent` metni HTML olarak **kodlayarak** yazar (`H03-detay.md`'deki konu). **Neden `@(120.50m)`?** Tırnak içindeki değer bir C# literali olarak derlenir; `120.50` bir `double`'dır, `decimal` özelliğe atamak için `m` eki gerekir. Düz metin alan attribute'lerde (`para-birimi` gibi) bu sorun yoktur.

5.3. `[HtmlTargetElement]` — var olan etikete bağlanmak

Tag Helper yeni bir etiket icat etmek zorunda değildir. `[HtmlTargetElement]` ile **bir attribute taşıyan her etikete** bağlanabilir:

```
[HtmlTargetElement(Attributes = "bold")]
public class BoldTagHelper : TagHelper
{
    public override void Process(TagHelperContext context, TagHelperOutput output)
    {
        output.Attributes.RemoveAll("bold");
        output.PreContent.SetHtmlContent("<strong>");
        output.PostContent.SetHtmlContent("</strong>");
    }
}

[HtmlTargetElement(Attributes = nameof(Condition))]
public class ConditionTagHelper : TagHelper
{
    public bool Condition { get; set; }

    public override void Process(TagHelperContext context, TagHelperOutput output)
    {
        if (!Condition)
            output.SuppressOutput();
    }
}
```

Yazılan	Üretilen
`<p bold>Kalın yazı</p>`	`<p><strong>Kalın yazı</strong></p>`
`<bold>Kalın mı?</bold>`	`<bold>Kalın mı?</bold>` — **dokunulmadı**
`<div condition="true">Görünür</div>`	`<div>Görünür</div>`
`<div condition="false">Görünmez</div>`	(hiçbir şey)

Üç gözlem:

- `RemoveAll("bold")` bölüm 1.2'deki teşhis kuralının ta kendisidir: **çalışan Tag Helper kendi attribute'ünü siler.** Bu kez o satırı biz yazdık.
- `[HtmlTargetElement]` yazıldığı anda sınıf adından gelen hedef (`<bold>`) **devre dışı kalır**. İkisini birden istiyorsanız iki ayrı `[HtmlTargetElement]` yazarsınız.
- `condition`, görünümdeki `@if`'in attribute ile yazılmış hâlidir. `false` olduğunda etiket içeriğiyle birlikte **hiç üretilmez** — Ctrl+U'da da yoktur.

5.4. Makinenin kolları: `TagHelperOutput`

Sunumdaki Tag Helper makinesinin istasyonları (`Match`, `Route lookup`, `RemoveAll()`, `SetAttribute()`) yukarıda yazdığımız satırların aynısıdır:

`TagHelperOutput`	Ne yapar	Nerede gördük
`TagName`	Etiketin adını değiştirir	`<email>` → `<a>`
`TagMode`	Kendini kapatan mı, açılış + kapanış mı	`FiyatTagHelper`
`Attributes.SetAttribute`	Attribute ekler ya da değiştirir	`href`, `class`

`Attributes.RemoveAll`	Attribute siler	`bold` — ve çatının `asp-*`'ları
`Content.SetContent`	İçeriği HTML olarak kodlayarak yazar	e-posta adresi, fiyat
`PreContent` / `PostContent`	İçeriğin önüne ve arkasına ekler	`<strong>`
`SuppressOutput()`	Hiçbir şey üretmez	`condition="false"`

Çatının `AnchorTagHelper`'ı da aynı sınıftan türer; tek farkı adresi rota tablosuna sormasıdır. Daha ileri adımlar (`ProcessAsync` ile içeriği okumak, `[HtmlAttributeName]`, Tag Helper mi View Component mi) `H05-detay.md` bölüm 1'de.

🐛 Hata Köşesi — Hafta 5

Belirti	Gerçek sebep	Çözüm
Sayfada `asp-for="…"` **yazısı görünüyor**	Tag Helper işlenmedi	`_ViewImports.cshtml`'de `@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers` satırı var mı?
`error CS1963: An expression tree may not contain a dynamic operation`	`asp-for` kullanılan görünümde `@model` **yok**	Görünümün başına `@model` yönergesini ekleyin
`error CS1061: … does not contain a definition for '…'`	`asp-for` değeri yanlış yazılmış	Özellik adını düzeltin — **bu hafta derleyici yakalıyor**
Form gönderiliyor ama **400** dönüyor	`[ValidateAntiForgeryToken]` var, form **elle** yazılmış	Formu `<form asp-action="…" method="post">` yapın
Token üretiliyor ama **hiç kontrol edilmiyor**	Action'da `[ValidateAntiForgeryToken]` yok	Attribute'ü ekleyin ya da `[AutoValidateAntiforgeryToken]` kullanın
Formda token **yok**	Form `method="get"`	GET'te token üretilmez; değiştiren işlem POST olmalı
Bağlantı **404** veriyor, dün çalışıyordu	`href` elle yazılmış, rota değişmiş	`asp-controller` / `asp-action` kullanın
`<select>` doğru seçeneği **işaretlemiyor**	`asp-for` yok, yalnızca `asp-items` var	`asp-for` ile modeldeki özelliği bağlayın
Onay kutusu hâlâ **false** geliyor	Form Tag Helper'ı devrede değil → gizli alan üretilmedi	`<input asp-for="…" />` kullanın
`type="number"` beklerken `text` geldi	Özellik `decimal`	Normaldir — ondalık ayracı yüzünden (bölüm 3.2)
Bağlantı **404**, hiç çalışmadı	`asp-action`'da yazım hatası; Tag Helper yine de adres üretir	Action adını kontrol edin (bölüm 2.5)

Kendi etiketiniz Ctrl+U'da **olduğu gibi** duruyor	Sınıf `public` değil, `TagHelper`'dan türemiyor ya da `@addTagHelper *, KampusKitap` yok	Üçünü sırayla kontrol edin (bölüm 5.1)
`<email … />` **yazısız** bir bağlantı üretti	Görünümde kendini kapatan yazım, Tag Helper'da `TagMode` yok	`output.TagMode = TagMode.StartTagAndEndTag;` (bölüm 5.2)
`<bold>` etiketi işlenmedi	`[HtmlTargetElement(Attributes = "bold")]` ad kuralını devre dışı bıraktı	İkinci bir `[HtmlTargetElement("bold")]` ekleyin (bölüm 5.3)
`dotnet run --environment Production` hâlâ Development	.NET 10'da `-e` bir ortam değişkeni ister	`dotnet run -e ASPNETCORE_ENVIRONMENT=Production` (bölüm 1.6)

🕳️ Klasik Tuzak — Adresi elle yazmak

```
<form action="/Kitap/Ekle" method="post">      @* ❌ *@
<a href="/Kitap/Detay/3">Detay</a>             @* ❌ *@
```

Bugün çalışır. İki şeyi birden kaybedersiniz:

1. **Rota değişince kırılır** — sessizce, 404 olarak.
2. **Anti-forgery token üretilmez** — form Tag Helper'ı devreye girmediği için.

İkinci madde daha sinsidir: POST action'ınıza sonradan `[ValidateAntiForgeryToken]` eklediğinizde form **400** vermeye başlar ve sebebi uzaktan görünmez.

```
<form asp-controller="Kitap" asp-action="Ekle" method="post">   @* ✅ *@
<a asp-controller="Kitap" asp-action="Detay" asp-route-id="3">Detay</a>   @* ✅ *@
```

📖 Terimler Sözlüğü

Terim	Ne demek
Tag Helper	Bir HTML etiketini sunucuda işleyip düz HTML üreten C# sınıfı
`@addTagHelper`	Hangi derlemedeki Tag Helper'ların kullanılacağını bildiren directive
`asp-for`	Bir alanı model özelliğine bağlayan attribute; `name`, `id`, `type`, `value` üretir
`asp-route-*`	URL'ye ek değer ekler; rota şablonuna göre yola ya da query string'e yazılır
`asp-items`	`<select>` seçeneklerinin kaynağı
`SelectList`	Kaynak listeyi `value`/metin çiftlerine dönüştüren sınıf

`[Display]`	Gösterilecek adı belirler — **doğrulama attribute'ü değildir**
`[DataType]`	Alanın anlamını belirtir; `type` çıkarımını etkiler
CSRF	Kullanıcının oturumunu kullanarak onun adına istek gönderten saldırı
Anti-forgery token	Sunucunun forma koyduğu, saldırganın okuyamayacağı gizli değer
`__RequestVerificationToken`	O token'ı taşıyan gizli alanın adı
`[ValidateAntiForgeryToken]`	Action'a gelen isteğin token'ını doğrulatan attribute
`<environment>`	İçeriğini yalnızca belirtilen ortamda üretir
`<cache>`	İçeriğinin çıktısını sunucu belleğinde belirli bir süre tutar
`asp-append-version`	Dosya adresine içerikten hesaplanan `?v=…` ekler
`asp-fallback-*`	Asıl CSS/JS dosyası gelmezse yedeğini yükleten sınama betiğini üretir
Form Action Tag Helper	`<button>` üzerindeki `asp-action` ile düğmenin `formaction`'ını üretir
`TagHelper` / `Process`	Kendi Tag Helper'ımızın türediği sınıf ve dönüşümü yapan metot
`TagHelperOutput`	`Process`'te değiştirilen çıktı: `TagName`, `TagMode`, `Attributes`, `Content`
`[HtmlTargetElement]`	Tag Helper'ın hangi etikete ya da attribute'e bağlanacağını belirler
kebab-case	`ParaBirimi` → `para-birimi`; özellik adının attribute adına dönüşme biçimi

---

✅ Kontrol Soruları

A. Tag Helper nedir

1. Tarayıcıya giden kaynakta `asp-for="Baslik"` yazısını görüyorsunuz. Ne ters gitti?
2. `@addTagHelper` satırı silinirse sayfa hangi durum kodunu döner? Bu neden tehlikelidir?
3. `@Html.TextBoxFor(...)` ile `<input asp-for="...">` arasındaki fark üretilen HTML'de mi, yazımda mı?
4. `@Html` yardımcılarına hâlâ ihtiyaç duyduğumuz bir yer söyleyin.
5. `<environment exclude="Development">` içindeki blok Development'ta Ctrl+U'da görünür mü? CSS ile gizlemekten farkı ne?
6. `<partial name="_KitapAlan" for="OneCikan" />`, partial'daki `<input asp-for="Baslik" />`'un `name`'ini nasıl değiştirir? `model=` ile yazılsaydı?

B. URL üretimi

7. `asp-route-id="3"` neden yola, `asp-route-kelime="suc"` neden query string'e yazıldı?
8. `asp-controller` yazılmazsa çatı hangi controller'ı varsayar?
9. Rota şablonunun başına `site/` eklendi. Elle yazılmış `href="/Kitap/Detay/3"` ne döner? `asp-action` ile üretilen?
10. Elle adres yazmanın anti-forgery ile ne ilgisi var?
11. `asp-action="Detya"` yazım hatası derlemede yakalanır mı? Üretilen adres ne döner? `asp-for`'dan farkı ne?

C. Form Tag Helper'ları

12. `<input asp-for="Fiyat" />` niçin `type="number"` değil `type="text"` üretiyor?
13. `bool` bir özellik için kaç `<input>` üretilir? İkincisi ne işe yarar?
14. `asp-for="Adres.Sehir"` hangi `name` ve hangi `id` değerini üretir? Neden farklılar?
15. `[Display(Name = "…")]` bir doğrulama attribute'ü müdür? Ne yapar?
16. `<select>` içine elle yazdığınız `<option>` silinir mi? Özellik `List<int>` olursa ne değişir?
17. `data-val-*` attribute'lerini bu hafta ne okuyor?
18. `string` ile `string?` arasındaki üretim farkını hangi C# özelliği doğuruyor?

D. CSRF

19. Saldırganın sayfası cookie'nizi gönderebiliyorsa, token'ı neden gönderemiyor?
20. Token'ı üretmek tek başına yeterli mi? Token'sız bir POST hangi durum kodunu alır, ekranda ne görünür?
21. Aynı formu `method="get"` yaparsanız token üretilir mi? Neden?
22. "Silme bağlantısı" olarak `<a href="/Kitap/Sil/3">` yazmak neden tehlikelidir?

E. Kendi Tag Helper'ınız

23. `KategoriRozetiTagHelper` sınıfı görünümde hangi etiketle, `KategoriId` özelliği hangi attribute'le yazılır?
24. `<email mail-to="Support" />` neden yazısız bir bağlantı üretti? Hangi satır çözer?
25. `[HtmlTargetElement(Attributes = "bold")]` yazılınca `<bold>` etiketi neden işlenmedi? `RemoveAll("bold")` satırı neden var?

📑 Kaynakça

> Her kaynağın yanında **bu ders notunun hangi bölümünü** dayandırdığı yazılıdır. Bağlantıların sonundaki `?view=aspnetcore-10.0` **kasıtlıdır**: sürüm sabitlenmezse arama motoru sizi eski sürüme düşürür. **Örnekte `Startup.cs` görüyorsanız yanlış sürümdesiniz.**

Zorunlu okuma

Kaynak	Dayandırdığı bölüm
Tag Helpers in ASP.NET Core — https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/intro?view=aspnetcore-10.0	Bölüm 1 tamamı — Tag Helper nedir, `@addTagHelper`, `@Html` ile karşılaştırma
Tag Helpers in forms in ASP.NET Core — https://learn.microsoft.com/aspnet/core/mvc/views/working-with-forms?view=aspnetcore-10.0	Bölüm 3 tamamı — `asp-for`, `label`, `textarea`, `select`

The Select Tag Helper — https://learn.microsoft.com/aspnet/core/mvc/views/working-with-forms?view=aspnetcore-10.0#the-select-tag-helper	Bölüm 3.4 — `asp-items`, `SelectList`, seçili öğe
Anchor Tag Helper — https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/built-in/anchor-tag-helper?view=aspnetcore-10.0	Bölüm 2.1–2.5 — `asp-route-*`, `asp-route`, `asp-all-route-data`, `asp-fragment`, `asp-protocol`, `asp-host`; `SpeakerController` örneği
ASP.NET Core built-in Tag Helpers — https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/built-in/?view=aspnetcore-10.0	Bölüm 1.5 — 18 Tag Helper'ın listesi
Author Tag Helpers — https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/authoring?view=aspnetcore-10.0	Bölüm 5 tamamı — `EmailTagHelper`, `BoldTagHelper`, `ConditionTagHelper`, `TagMode`

Sayfa düzeyindeki ve diğer hazır Tag Helper'lar

Kaynak	Dayandırdığı bölüm
Environment Tag Helper — https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/built-in/environment-tag-helper?view=aspnetcore-10.0	Bölüm 1.6 — `include`, `exclude`, `names`
Cache Tag Helper — https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/built-in/cache-tag-helper?view=aspnetcore-10.0	Bölüm 1.6 — `expires-after`, `vary-by-*`
Distributed Cache Tag Helper — https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/built-in/distributed-cache-tag-helper?view=aspnetcore-10.0	Bölüm 1.6 — `name` anahtarı, depo
Image Tag Helper — https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/built-in/image-tag-helper?view=aspnetcore-10.0	Bölüm 1.6 — `asp-append-version`
Link Tag Helper — https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/built-in/link-tag-helper?view=aspnetcore-10.0	Bölüm 1.6 — `asp-fallback-href`, `asp-fallback-test-*`
Script Tag Helper — https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/built-in/script-tag-helper?view=aspnetcore-10.0	Bölüm 1.6 — `asp-fallback-src`, `asp-fallback-test`
Partial Tag Helper — https://learn.microsoft.com/aspnet/core/mvc/views/tag-helpers/built-in/partial-tag-helper?view=aspnetcore-10.0	Bölüm 1.6 — `model` ve `for`
The Form Action Tag Helper — https://learn.microsoft.com/aspnet/core/mvc/views/working-with-forms?view=aspnetcore-10.0#the-form-action-tag-helper	Bölüm 2.6 — `formaction`
The Validation Message Tag Helper — https://learn.microsoft.com/aspnet/core/mvc/views/working-with-forms?view=aspnetcore-10.0#the-validation-message-tag-helper	Bölüm 3.6 — `asp-validation-for` ve `asp-validation-summary`

Güvenlik

Kaynak	Dayandırdığı bölüm

Prevent XSRF/CSRF attacks in ASP.NET Core — https://learn.microsoft.com/aspnet/core/security/anti-request-forgery?view=aspnetcore-10.0	Bölüm 4 tamamı — token üretimi, doğrulama, `[AutoValidateAntiforgeryToken]`
Cross-Site Request Forgery (CSRF) (MDN) — https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/CSRF	Bölüm 4.1 — saldırının tarayıcı tarafındaki tanımı

Köprüler (sonraki haftalara)

Kaynak	Dayandırdığı bölüm
Model validation in ASP.NET Core MVC — https://learn.microsoft.com/aspnet/core/mvc/models/validation?view=aspnetcore-10.0	Bölüm 3.6 — `data-val-*` attribute'lerini kim okuyor (**Hafta 6**)

📚 Daha Fazlası

`H05-detay.md` dosyasında (**sınav kapsamı dışı**):

- Kendi Tag Helper'ınızda ileri adımlar: `ProcessAsync` ile içeriği okumak, `[HtmlAttributeName]`, `Order`; Tag Helper mi View Component mi
- Sayfa düzeyindeki Tag Helper'larda ayrıntılar: gerçek bir CDN ile `asp-fallback-*`, `<distributed-cache>`, `asp-area` ve `asp-page`
- `[AutoValidateAntiforgeryToken]` ve token'ı AJAX isteklerinde taşımak
- `SameSite` cookie attribute'ünün CSRF'e katkısı (Hafta 7 köprüsü)
- API'lerde CSRF neden farklı ele alınır (Hafta 12 köprüsü)
- Doğrulama mesajlarının Türkçeleştirilmesi

➡️ Haftaya Hazırlık

Bu hafta `data-val-required`, `data-val-length-max` gibi attribute'lerin HTML'e **kendiliğinden** yazıldığını gördük — ve onları **okuyan hiçbir şey olmadığını**.

Hafta 6'da o attribute'leri okuyan tarayıcı tarafını ve asıl önemlisi **sunucu tarafı doğrulamayı** kuracağız.

Düşünerek gelin: Tarayıcıdaki doğrulama kullanıcıya yardım eder, hatayı erken gösterir. Peki kullanıcı tarayıcının doğrulamasını **atlatırsa** ne olur? Hafta 4'ün ilkesi — *istemciden gelen hiçbir veri güvenilir değildir* — burada hâlâ geçerli mi?

> 🏠 Ev göreviniz bu sorunun zeminini hazırlıyor: Hafta 4'teki formunuzu Tag Helper'larla yeniden yazıp iki HTML çıktısını **yan yana** koyacaksınız. `data-val-*` satırlarını da işaretleyin — haftaya onlarla başlayacağız.